

TUGAS INDIVIDU APLIKASI QUEUE

Nama : Muhammad Akmal Fazli Riyadi

NIM : 24060124130123

Kelas : A

Proses 1: IDProses = A, BT = 3

Proses 2: IDProses = B, BT = 6

Proses 3: IDProses = C, BT = 1

Proses 4: IDProses = D, BT = 2

Proses 5: IDProses = E, BT = 3

IDProses	Burst Time	Waktu Mulai	Waktu Selesai
C	1	0	1
D	2	1	3
E	3	3	6
A	3	6	9
B	6	9	15

Program PenjadwalanProsesCPU

{program untuk mensimulasikan penjadwalan proses pada CPU
menggunakan algoritma penjadwalan shortest job first}

Kamus

use Proses

use Tqueue

Q: antrean proses di CPU

P1, P2, P3, P4, P5, prosesDieksekusi, temp: Proses
waktuSkrng, waktuMulai, waktuSelesai, i, j: integer

Algoritma

makeProses(P1, "A", 3)

makeProses(P2, "B", 6)

makeProses(P3, "C", 1)

makeProses(P4, "D", 2)

makeProses(P5, "E", 3)

createQueue(Q)

enqueue(Q, P1)

```

enqueue(Q, P2)
enqueue(Q, P3)
enqueue(Q, P4)
enqueue(Q, P5)

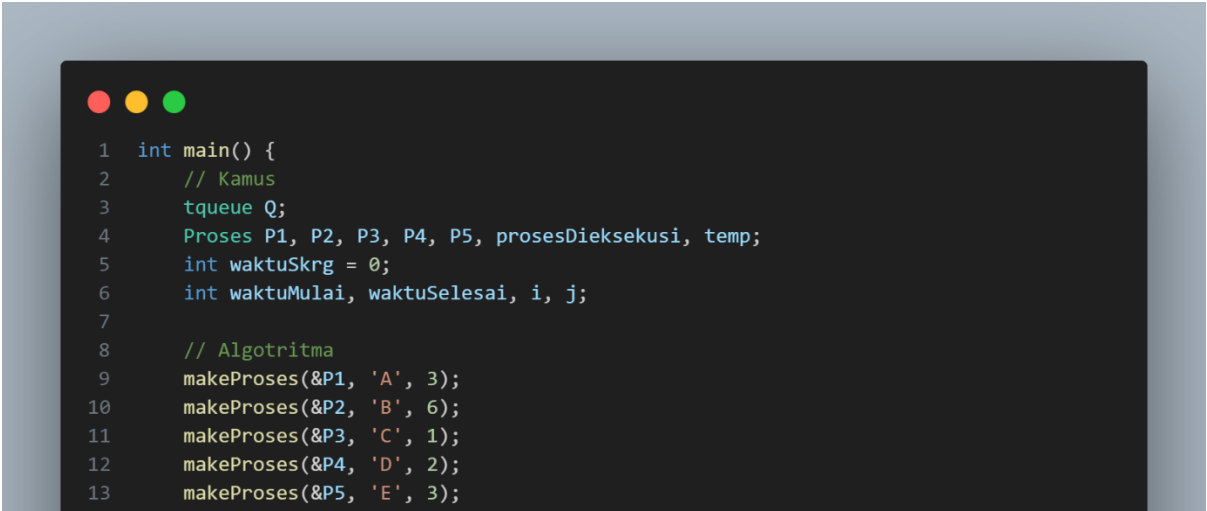
output("Keadaan queue awal:")
printQueue(Q)

output("Keadaan queue setelah diurutkan berdasarkan shortest
job first:")
i traversal [1 .. 4]
    j traversal [1 .. 5 - i]
        if getBurstTime(Q.wadah[j]) >
            getBurstTime(Q.wadah[j + 1]) then
            temp ← Q.wadah[j]
            Q.wadah[j] ← Q.wadah[j + 1]
            Q.wadah[j + 1] ← temp
    {end traversal}
{end traversal}
printQueue(Q)

output("Urutan eksekusi proses")
waktuSkrng ← 0
while not isEmptyQueue(Q) do
    waktuMulai ← waktuSkrng
    waktuSelesai ← waktuSkrng + getBurstTime(infoHead(Q))

    output(getIDProses(prosesSkrng), waktuMulai,
        waktuSelesai, getBurstTime(infoHead(Q))
    waktuSkrng ← waktuSelesai
    dequeue(Q, prosesDieksekusi)
{end while}

```



```

1  int main() {
2      // Kamus
3      tqueue Q;
4      Proses P1, P2, P3, P4, P5, prosesDieksekusi, temp;
5      int waktuSkrng = 0;
6      int waktuMulai, waktuSelesai, i, j;
7
8      // Algoritma
9      makeProses(&P1, 'A', 3);
10     makeProses(&P2, 'B', 6);
11     makeProses(&P3, 'C', 1);
12     makeProses(&P4, 'D', 2);
13     makeProses(&P5, 'E', 3);

```

```

14
15     createQueue(&Q);
16
17     enqueue(&Q, P1);
18     enqueue(&Q, P2);
19     enqueue(&Q, P3);
20     enqueue(&Q, P4);
21     enqueue(&Q, P5);
22
23     printf("Keadaan queue awal:\n");
24     printQueue(&Q);
25
26     printf("\n\nKeadaan queue setelah diurutkan berdasarkan shortest job first:\n");
27     for (i = 1; i <= 5 - 1; i++) {
28         for (j = 1; j <= 5 - i; j++) {
29             if (getBurstTime(Q.wadah[j]) > getBurstTime(Q.wadah[j + 1])) {
30                 temp = Q.wadah[j];
31                 Q.wadah[j] = Q.wadah[j + 1];
32                 Q.wadah[j + 1] = temp;
33             }
34         }
35     }
36     printQueue(&Q);
37
38     printf("\n\nUrutan eksekusi proses:\n");
39     while (!isEmptyQueue(Q)) {
40         waktuMulai = waktuSkrng;
41         waktuSelesai = waktuSkrng + getBurstTime(infoHead(Q));
42         printf("Eksekusi Proses '%c', Mulai: %d, Selesai: %d\n",
43             getIDProses(infoHead(Q)),
44             waktuMulai,
45             waktuSelesai);
46
47         waktuSkrng = waktuSelesai;
48         dequeue(&Q, &prosesDieksekusi);
49     }
50     return 0;
51 }

```

Keadaan queue awal:

ID: A | BT: 3
ID: B | BT: 6
ID: C | BT: 1
ID: D | BT: 2
ID: E | BT: 3

Keadaan queue setelah diurutkan berdasarkan shortest job first:

ID: C | BT: 1
ID: D | BT: 2
ID: A | BT: 3
ID: E | BT: 3
ID: B | BT: 6

Urutan eksekusi proses:

Eksekusi Proses 'C', Mulai: 0, Selesai: 1
Eksekusi Proses 'D', Mulai: 1, Selesai: 3
Eksekusi Proses 'A', Mulai: 3, Selesai: 6
Eksekusi Proses 'E', Mulai: 6, Selesai: 9
Eksekusi Proses 'B', Mulai: 9, Selesai: 15